process to denote the implementing class as one that contains route model conventions that should be applied to the application.

The `PageRouteModel` parameter provides a gateway for applying new conventions to generate routes for a Razor page. It is through this object that you can apply your custom convention. The `PageRouteModel` has a `Selectors` property, which represents a collection of `SelectorModel` objects. Each one of these has an `AttributeRouteModel` property, which, in turn, has a `Template` property representing a route template that enables mapping a URL to this particular page. The following is how this structure looks for the current Index page in our application:

```
PageRouteModel
    RelativePath: "/Pages/Index.cshtml"
    Selectors: [Count = 3]
        SelectorModel[0]:
            AttributeRouteModel:
                Template: "Index"
        SelectorModel[1]:
            AttributeRouteModel:
                Template: ""
        SelectorModel[2]:
            AttributeRouteModel:
                Template: "FindMe"
```

This is a massively simplified representation, focusing only on those properties that are of immediate interest. The `PageRouteModel` class and the classes that make up its properties are a lot more complex than this. Notice that there are three `SelectorModel` objects in this representation of the Index page. The final `SelectorModel` contains the `FindMe` route template, which was added in the previous section through the `AddPageRoute` method.